Intro



안녕하세요, 개발자 최두용 입니다.

1996. 10. 03

chlendyd7@gmail.com

010-4632-6222

https://github.com/chlendyd7

Summary

AI가 코드를 짜는 시대이지만, 여전히 개발자는 비즈니스 요구사항을 정확히 정의하고 문제 해결의 방향성을 제시해야 한다고 생각합니다.
복잡한 프로세스를 도식화하여 놓치기 쉬운 예외를 보완하고, 체계적인 문서화로 유지보수와 AI 활용에 용이한 시스템을 만듭니다.

핵심 역량

성능 개선

개발자로써 한정 된 자원 속 최선의 결과를 내기 위해 노력합니다

N+1, 비동기, 캐싱 등 기법을 사용해 응답 속도를 30% 이상 개선한 경험이 있습니다

요구사항 분석·구조화

MES 개발 당시 ERP·WMS 등 여러 시스템이 얽히거나 복잡한 프로세스는 도식화하여 진행했습니다

구두로 설명하기 프로세스를 흐름도와 인터페이스 정의서로 정리해 이를 기준으로 예외사항을 놓치지 않으며 개발을 진행했습니다

문서화

기억보다 기록이 더 강력하는 것을 알기에 회의록, 테이블 설계서 등 문서를 작성합니다

만들어진 문서를 유지보수 및 AI 활용에 사용하여 더 좋은 결과물을 창출합니다

경력

현대 위아 2026.04 ~ ing

시스템 유지보수 및 개발

- 열관리 시스템 MES 유지보수 (Spring, C#, Vue)
- SCADA 시스템 PL(협력사 일정 조율 및 산출물 관리)

애기야가자 2023. 11. 20 ~ 2024. 08. 15

서버 및 백엔드 개발

- 200MAU 백엔드 및 서버 운영
- N+1, 캐싱, 비동기 처리로 성능 30 ~ 80% 개선

HnK 株式会社 2023. 05. 01 ~ 2023. 11. 20

백엔드 및 프론트 개발 (프리랜서)

- 일본 의료 동영상 웹사이트 구현
- 게시판 기능, 결제 자동화, 실시간 채팅 등 구현

주요 기술

업무에 사용

- Spring Boot, C#, Vue, Django, ProstreSQL, Redis, NoSQL

과제에 사용

- Kubernetes, Jenkins, Kafka, k6, MCP

학습

- LangChain, LangGraph, RAG, Rancher, AWS

Project

부하 테스트에서 WebSocket 연결 실패율 44%를 발견하고, 병목 원인을 분석해 0%로 해결 - 호가창 프로젝트

문제 원인

- 대용량 데이터를 위한 봇 주문 생성 로직에서의 과도한 데이터 Push
- 체결 내역 조회 기능이 아이템 별 스케줄링 동작
- WebSocket으로 전환하며 cpu, network 사용량이 늘어나자 보여진 기존 프로세스 문제점들과의 충돌

해결 과정

- 한번에 생성하던 대용량 봇 주문 생성을 ms 단위로 분리하여 stream으로 동일 수량의 주문을 시간축으로 분산, 스파이크 부하를 낮춤
- 스케줄링을 통해 아이템을 순차 확인하던 로직을 멀티 스레드로 바꿔 각 아이템을 한번에 조회해 Lua 스크립트와 연계, 데이터 적재량을 낮춤
- 테스트
  - k6 Smoke, Load, Stress test 진행
  - 개발자 도구를 이용해 Network tab 데이터 속도 수동확인

결과

- 200 vu stress test 재측정 결과 WebSocket 연결 실패율 44% → 0% 달성 확인
- CPU 사용률과 데이터 적재량의 감소로 리소스 안정성 향상, 해당 이슈가 사라짐
- 그러나 HPA 스케일 아웃으로 레플리카가 늘어나자 단일 인스턴스를 전제로 작성된 스케줄러봇이 Pod 수만큼 중복 실행
  MSA 분리는 시간 공수 제한으로 롤백, ShedLock을 적용해 한 시점에 한 Pod에서만 실행되도록 제어

상세 화면 API 지연을, 서버 사양 업그레이드 대신 쿼리와 저장 구조 개선으로 해결했습니다

문제 원인

- 사용자가 늘면서 상세 화면 API가 느려졌다는 피드백을 받았습니다.
- ORM을 사용하여 N+1 쿼리가 일어나고 있는 상황
- 상세 화면 접속 시 조회 수를 유저 - 상세화면 N:M 테이블을 매번 계산

해결 과정

- Debugger를 이용해 쿼리를 분석해 N+1을 먼저 해결
- 반정규화를 통해 조회 수 칼럼을 상세화면 테이블에 추가
- 인기 장소 상세 내역을 DynamoDB에 저장해 캐시처럼 사용 (Redis는 Celery 메시지 큐로 쓰고 있어, 시니어 개발자의 의견에 따라 NoSQL을 선택)
- 테스트
  - 개발자 도구와 TTFB와 Finish 시간을 측정
  - Debugger로 DB 1 hit 확인

결과

- 병목이 있던 상세 화면 800ms에서 300ms로, 인기 장소 129ms에서 24ms로 줄임
- 고려하던 서버 사양 업그레이드 대신 쿼리 개선과 DB 튜닝, DyanmoDB로 저렴하게 문제 해결

라인을 가동하지 않아도 PPC 시스템 데이터 입력·전송을 검증할 수 있는 테스트 경로를 만들었습니다

문제 원인

- 기존 현장 단말(PPC)은 라인과 결합도가 높아 라인을 가동하지 않으면 데이터가 제대로 입력되고 전송되는지 확인할 수 없는 상황
- 라인이 멈추는 주말에 미니 PC를 교체했는데, 그 시점에 시스템을 테스트할 방법이 없었음
- 교체 후 정상 동작 여부를 확인하지 못해 불안했고, 이 부분을 개선하고자 함

해결 과정

- I/F 자료를 분석해 어떤 데이터가 들어오고 라인 데이터가 어디에 적재 되는지 먼저 확인
- 버튼 하나로 테스트 데이터가 화면에 입력되도록 구성하였고 데이터를 적재하는 테스트 테이블을 설계
- PLC 네트워크를 설정하면 통신 포트로 Socket 연결을 시도해 "연결됨/연결 안 됨"으로 표시 기능 추가

결과

- 라인을 가동하지 않아도 데이터 입력과 적재가 정상인지, PLC 네트워크에 도달하는지 확인할 수 있게 됨
- 바코드 프린터 출력 테스트까지 확인할 수 있음
- 해당 시스템 외에도 타 시스템이나 라인 결합도가 높은 시스템이 발견되면 테스트를 할 수 있게 구성 하고자 함